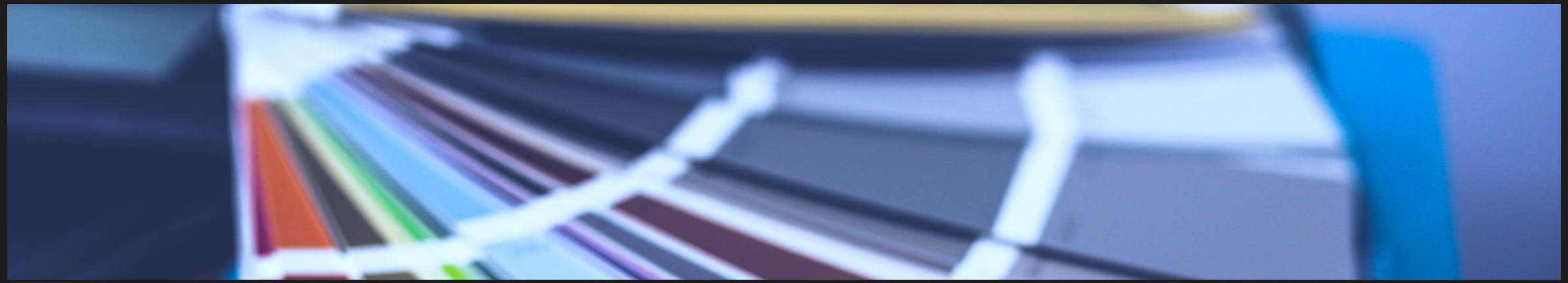


Exkurs E

Entwurfsmuster

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Agenda

Abschnitt E.1
Einführung zu Entwurfsmustern

Abschnitt E.2
Factory Method (Fabrikmethode)

Abschnitt E.3
State (Zustand)

Abschnitt E.4
Command (Befehl)

Abschnitt E.5
Memento

Abschnitt E.6
Observer (Beobachter)

Abschnitt E.7
Strategy (Strategie)

Abschnitt E.8
Decorator (Dekorierer)

Klausurrelevante Lernziele

- Klassifikation der Entwurfsmuster als Erzeugungs-, Struktur- und Verhaltensmuster erläutern und jeweils zwei Beispiele nennen können
- Anwendungsfälle, Ziele und einige Vor- bzw. Nachteile der folgenden Entwurfsmuster erläutern sowie die Schemata auf ein Beispiel anwenden können:
  - Factory Method
  - State
  - Command
  - Observer
  - Strategy

Abschnitt E.1

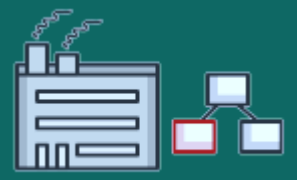
Einführung zu Entwurfsmustern

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Was sind Entwurfsmuster?

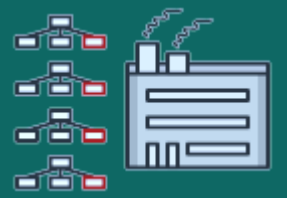
- Entwurfsmuster (Design Patterns)
  - Bewährte Lösungsschablonen für wiederkehrende Entwurfsprobleme
  - Sprachunabhängig, aber im Allgemeinen objektorientiert
- Historisch: Gang of Four (GoF, 1994)
  - Erich Gamma, Richard Helm, Ralph E. Johnson, John Vlissides
- Klassifikation nach Zweck
  - Erzeugungsmuster (Creational Patterns): Erstellungsprozess von Objekten
  - Strukturmuster (Structural Patterns): Zusammensetzung von Klassen und Objekten
  - Verhaltensmuster (Behavioral Patterns): Interaktion von Klassen und Objekten sowie die Verteilung von Zuständigkeiten
- Klassifikation nach Gültigkeitsbereich
  - Klassenbasiert: Beziehungen von Klassen zu ihren abgeleiteten Klassen (bei Kompilierung gemäß Vererbung)
  - Objektbasiert: Beziehungen zwischen Objekten zur Laufzeit

Erzeugungsmuster (1)



Factory Method (Fabrikmethode)

Definition einer Schnittstelle zur Objekterzeugung, wobei die Bestimmung der zu instanziierenden Klasse den Unterklassen überlassen bleibt. Es gestattet einer Klasse, die Instanziierung an Unterklassen zu delegieren.

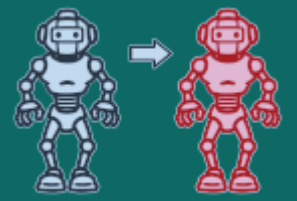


Abstract Factory (Abstrakte Fabrik)

Bereitstellung einer Schnittstelle zum Erzeugen verwandter oder voneinander abhängiger Objektfamilien ohne die Benennung ihrer konkreten Klassen.

Builder (Erbauer)

Getrennte Handhabung der Erzeugungs- und Darstellungsmechanismen komplexer Objekte zwecks Generierung verschiedener Repräsentationen in einem einzigen Erzeugungsprozess.



Prototype (Prototyp)

Spezifikation der zu erzeugenden Objekttypen mittels einer prototypischen Instanz und Erzeugung neuer Objekte durch Kopieren dieses Prototyps.

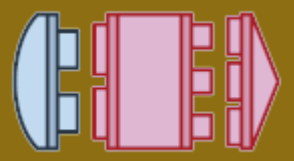
Erzeugungsmuster (2)

Singleton (Einzelstück)

Sicherstellung der Existenz nur einer einzigen Klasseninstanz sowie Bereitstellung eines globalen Zugriffspunkts für diese Instanz.

- Factory Method (Fabrikmethode) ist klassenbasiert, alle anderen objektbasiert

Strukturmuster (1)



Adapter (Adapter)

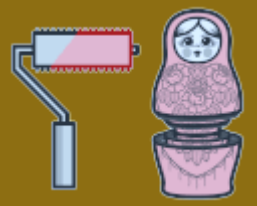
Anpassung der Schnittstelle einer Klasse an ein anderes von den Clients erwartetes Interface. Es ermöglicht die Zusammenarbeit von Klassen, die ansonsten aufgrund der Inkompatibilität ihrer Schnittstellen nicht dazu in der Lage wären.

Bridge (Brücke)

Entkopplung einer Abstraktion von ihrer Implementierung, so dass beide unabhängig voneinander variiert werden können.

Composite (Kompositum)

Komposition von Objekten in Baumstrukturen zur Abbildung von Teil-Ganzes-Hierarchien. Es gestattet den Clients einen einheitlichen Umgang sowohl mit individuellen Objekten als auch mit Objektkompositionen.



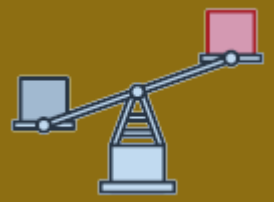
Decorator (Dekorierer)

Dynamische Erweiterung der Funktionalität eines Objekts. Decorator-Objekte stellen hinsichtlich der Ergänzung einer Klasse um weitere Zuständigkeiten eine flexible Alternative zur Unterklassenbildung dar.

Strukturmuster (2)

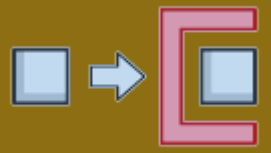
Façade (Fassade)

Bereitstellung einer einheitlichen Schnittstelle zu einem Schnittstellensatz in einem Subsystem. Es definiert eine Schnittstelle höherer Ebene, die die Nutzung des Subsystems vereinfacht.



Flyweight (Fliegengewicht)

Gemeinsame Nutzung feingranularer Objekte, um sie auch in großer Anzahl effizient nutzen zu können.

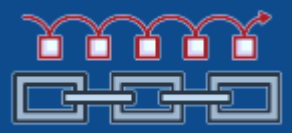


Proxy (Proxy)

Bereitstellung eines vorgelagerten Stellvertreterobjekts bzw. eines Platzhalters zwecks Zugriffssteuerung eines Objekts.

- Von Adapter (Adapter) gibt es eine klassen- und eine objektbasierte Variante

Verhaltensmuster (1)



Chain of Responsibility (Zuständigkeitskette)

Vermeidung der Kopplung eines Request-Auslösers mit seinem Empfänger, indem mehr als ein Objekt in die Lage versetzt wird, den Request zu bearbeiten. Der Request wird so lange entlang einer Kette von Objekten weitergeleitet, bis er von einem Objekt bearbeitet wird.

Command (Befehl)

Kapselung eines Requests als Objekt, um so die Parametrisierung von Clients mit verschiedenen Requests, Warteschlangen- oder Logging-Operationen sowie das Rückgängigmachen von Operationen zu ermöglichen.

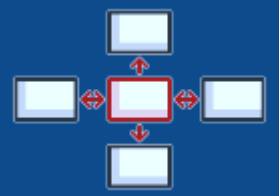
Interpreter (Interpreter)

Definition einer Repräsentation für die Grammatik einer gegebenen Sprache und Bereitstellung eines Interpreters, der sie nutzt, um in dieser Sprache verfasste Sätze zu interpretieren.

Iterator (Iterator)

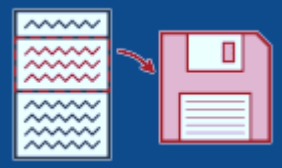
Bereitstellung eines sequenziellen Zugriffs auf die Elemente eines aggregierten Objekts, ohne dessen zugrunde liegende Struktur offenzulegen.

Verhaltensmuster (2)



Mediator (Vermittler)

Definition eines Objekts, das die Interaktionsweise eines Objektsatzes in sich kapselt. Es begünstigt lose Kopplungen, indem es die explizite Referenzierung der Objekte untereinander unterbindet und so eine individuelle Steuerung ihrer Interaktionen ermöglicht.

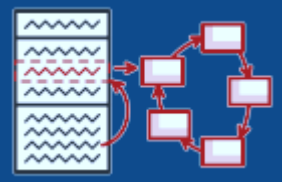


Memento (Memento)

Erfassung und Externalisierung des internen Zustands eines Objekts, ohne dessen Kapselung zu beeinträchtigen, so dass es später wieder in diesen Zustand zurückversetzt werden kann.

Observer (Beobachter)

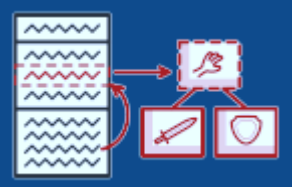
Definition einer 1-zu-n-Abhängigkeit zwischen Objekten, damit im Fall einer Zustands-änderung eines Objekts alle davon abhängigen Objekte entsprechend benachrichtigt und automatisch aktualisiert werden.



State (Zustand)

Anpassung der Verhaltensweise eines Objekts im Fall einer internen Zustandsänderung, so dass es den Anschein hat, als hätte es seine Klasse gewechselt.

Verhaltensmuster (3)

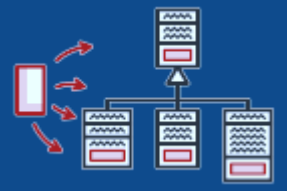


Strategy (Strategie)

Definition einer Familie von einzeln gekapselten, austauschbaren Algorithmen. Es ermöglicht eine variable und von den Clients unabhängige Nutzung des Algorithmus.

Template Method (Schablonenmethode)

Definition der Grundstruktur eines Algorithmus in einer Operation sowie Delegation einiger Ablaufschritte an Unterklassen. Es ermöglicht den Unterklassen, bestimmte Schritte eines Algorithmus zu überschreiben, ohne dessen grundlegende Struktur zu verändern.

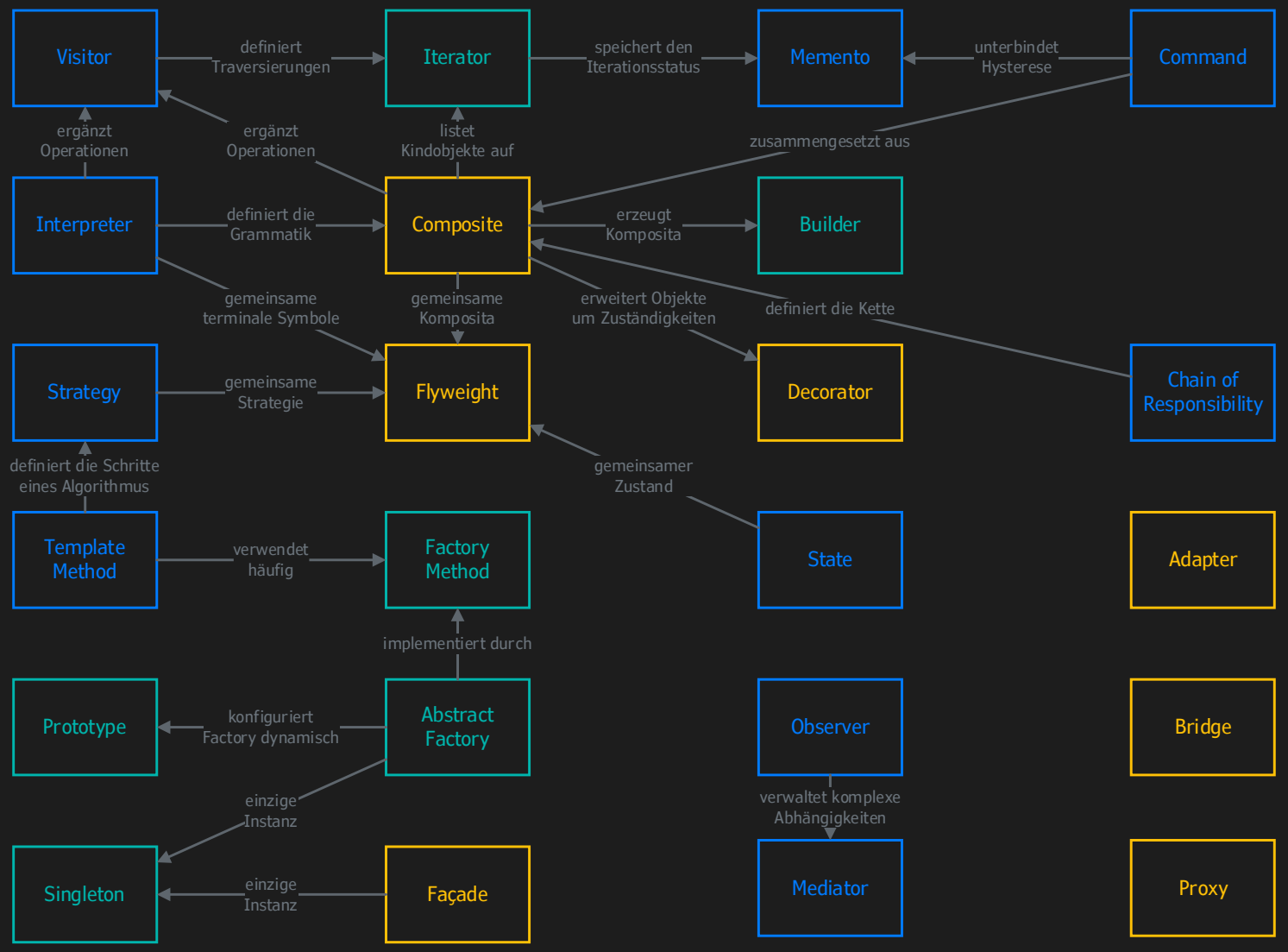


Visitor (Besucher)

Darstellung einer auf die Elemente einer Objektstruktur anzuwendenden Operation. Es ermöglicht die Definition einer neuen Operation, ohne die Klassen der von ihr bearbeiteten Elemente zu verändern.

- Interpreter (Interpreter) und Template Method (Schablonenmethode) sind klassenbasiert, alle anderen objektbasiert

Beziehungen zwischen den Entwurfsmustern



Quelle: nach (Gamma, Helm, Johnson, Vlissides 2014)

Abschnitt E.2

Factory Method (Fabrikmethode)

Erzeugungsmuster: Factory Method

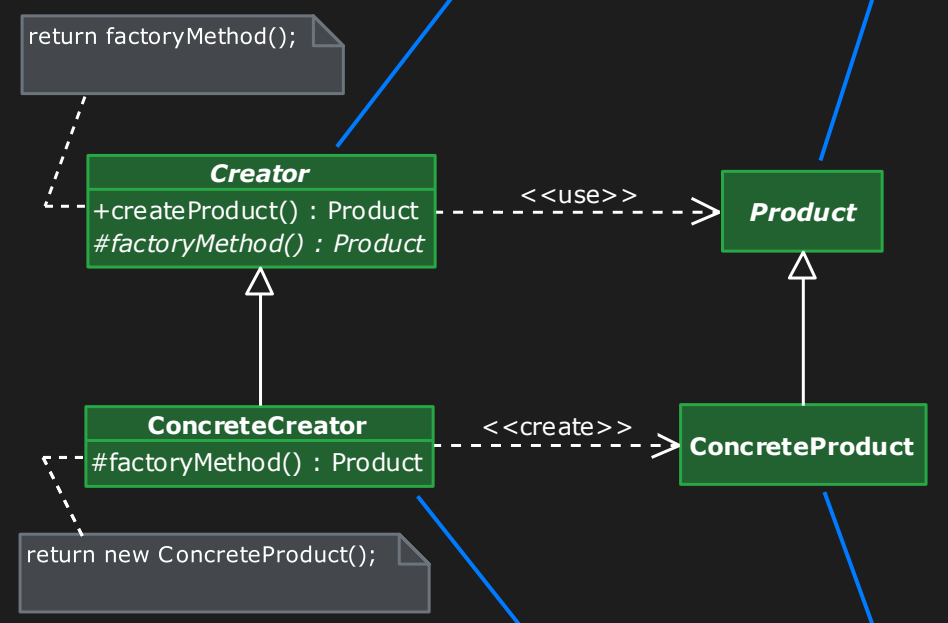
Factory Method – Definition einer Schnittstelle zur Objekterzeugung, wobei die Bestimmung der zu instanziierenden Klasse den Unterklassen überlassen bleibt. Es gestattet einer Klasse, die Instanziierung an Unterklassen zu delegieren.

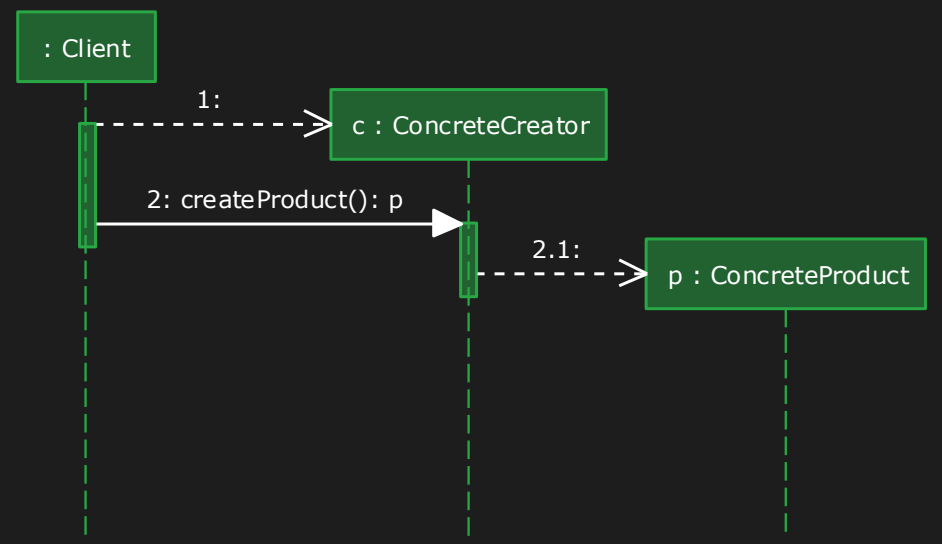
- Anwendungsfälle
  - Wenn eine Klasse Objekte erzeugen muss, deren Datentyp und Abhängigkeiten im Vorhinein nicht (vollständig) bestimmt werden kann
  - Wenn eine Klasse von Ihren Unterklassen eine Spezifizierung der erzeugten Objekte erwartet
  - Wenn eine Klasse Zuständigkeiten an eine von mehreren Unterklassen delegiert und festgestellt werden soll, welche Unterklasse genutzt wird
  - Wenn mit einer wachsenden Anzahl von verschiedenartigen »Produkten« zu rechnen ist, die alle einen gemeinsamen Herstellungsprozess durchlaufen müssen, der zentral gesteuert und unabhängig von den Konstruktoren der Produkte sein soll
  - Kann auch genutzt werden, um bereits vorhandene Objekte wiederzuverwenden, statt sie jedes Mal neu zu erzeugen

Factory Method: Schema & Sequenz

Deklariert die Factory-Methode, die neue Objekt des Typs Product zurück-gibt; stellt einheitliche Geschäftslogik des Herstellungsprozesses sicher

Gemeinsame Schnittstelle aller Objekte, die durch die Factory-Methode erzeugt werden können

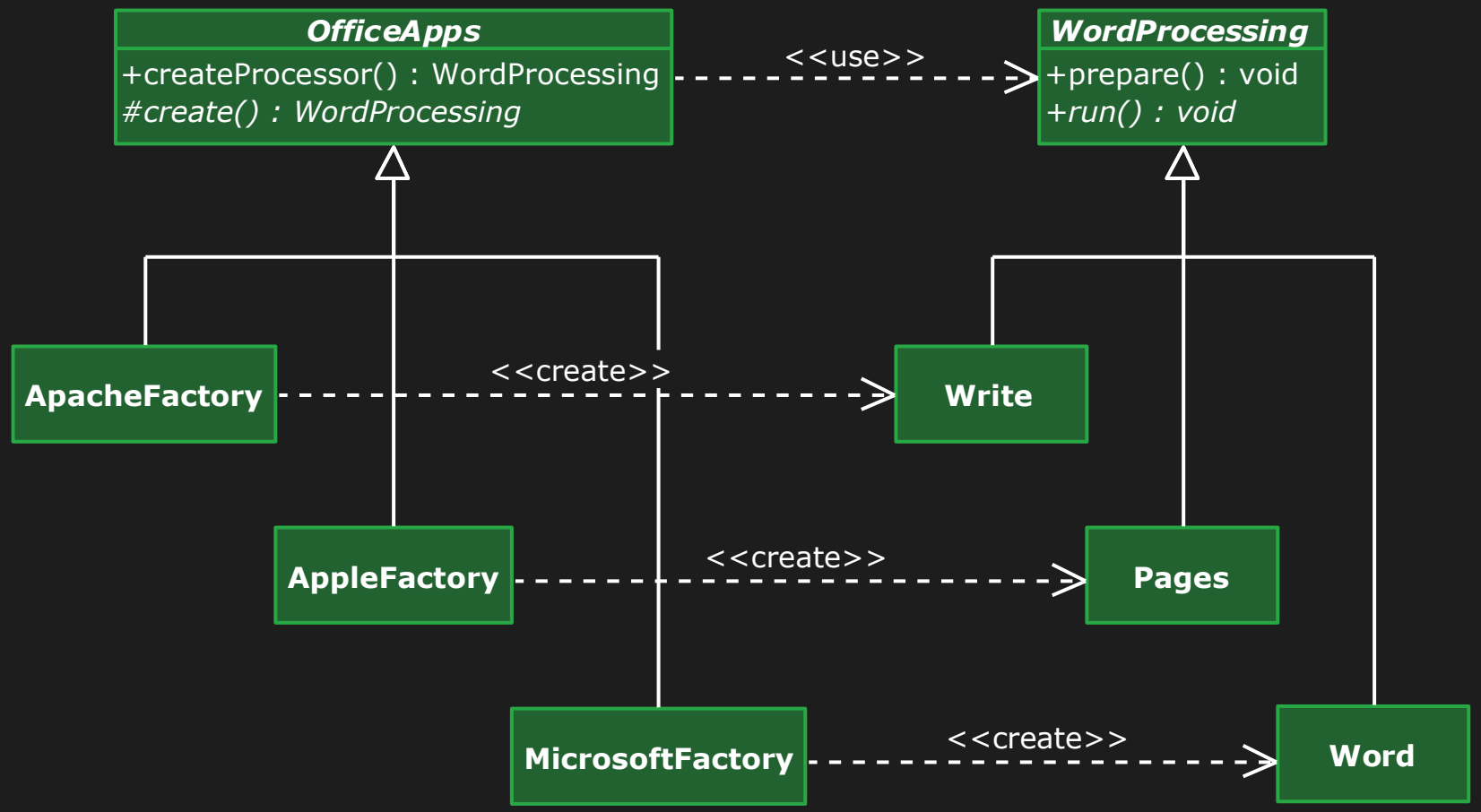




Überschreibt die Factory-Methode, um eine Instanz von ConcreteProduct zurückzugeben

Konkrete Implementierung der Product-Schnittstelle

Factory Method: Beispiel



Factory Method: Vor- und Nachteile

Vorteile

- Herstellungsprozess ist von konkreter Implementierung getrennt [→ SRP]
- Unterschiedliche Produkte durchlaufen denselben Herstellungsprozess
- Wiederverwendbarkeit & Konsistenz der Herstellung
- Neue Produkte können ohne Anpassung von altem Code ergänzt werden [→ OCP]
- Verbindung paralleler Klassen-hierarchien

Nachteile

- Code wird komplizierter: es sind einige neue Klassen nötig
- Enge Kopplung von konkretem Hersteller und konkretem Produkt

Abschnitt E.3

State (Zustand)

Verhaltensmuster: State

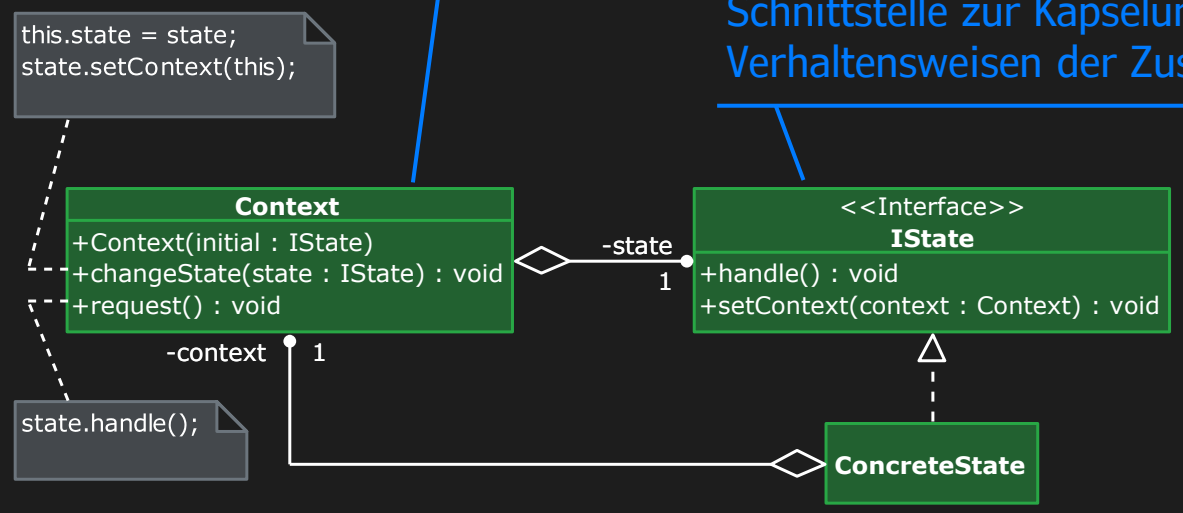
State – Anpassung der Verhaltensweise eines Objekts im Fall einer internen Zustandsänderung, so dass es den Anschein hat, als hätte es seine Klasse gewechselt.

- Anwendungsfälle
  - Verhalten eines Objekts hängt von seinem Zustand ab, der sich zur Laufzeit ändern kann; dabei gibt es viele Zustände und alle haben ein spezifisches Verhalten
  - Objektorientierte und flexible Alternative zu Zustandskonstanten bzw. Konstruktionen mit Enums & Switch für interne Zustände
  - Implementierung von Objekten & Zuständen aus UML-Aktivitäts- bzw. Zustandsdiagramm
- Beispiele
  - TCP-Connection: Established, Listening, Closed
  - Git-Lebenszyklus von Dateien: Untracked, Staged, Modified, Committed
  - Urlaubsantrag: Neu, In Prüfung, Abgelehnt, Genehmigt, Abgeschlossen

State: Schema

Schnittstelle für Clients; speichert die Referenz auf den aktuellen Zustand; delegiert die Bearbeitung an den aktuellen Zustand

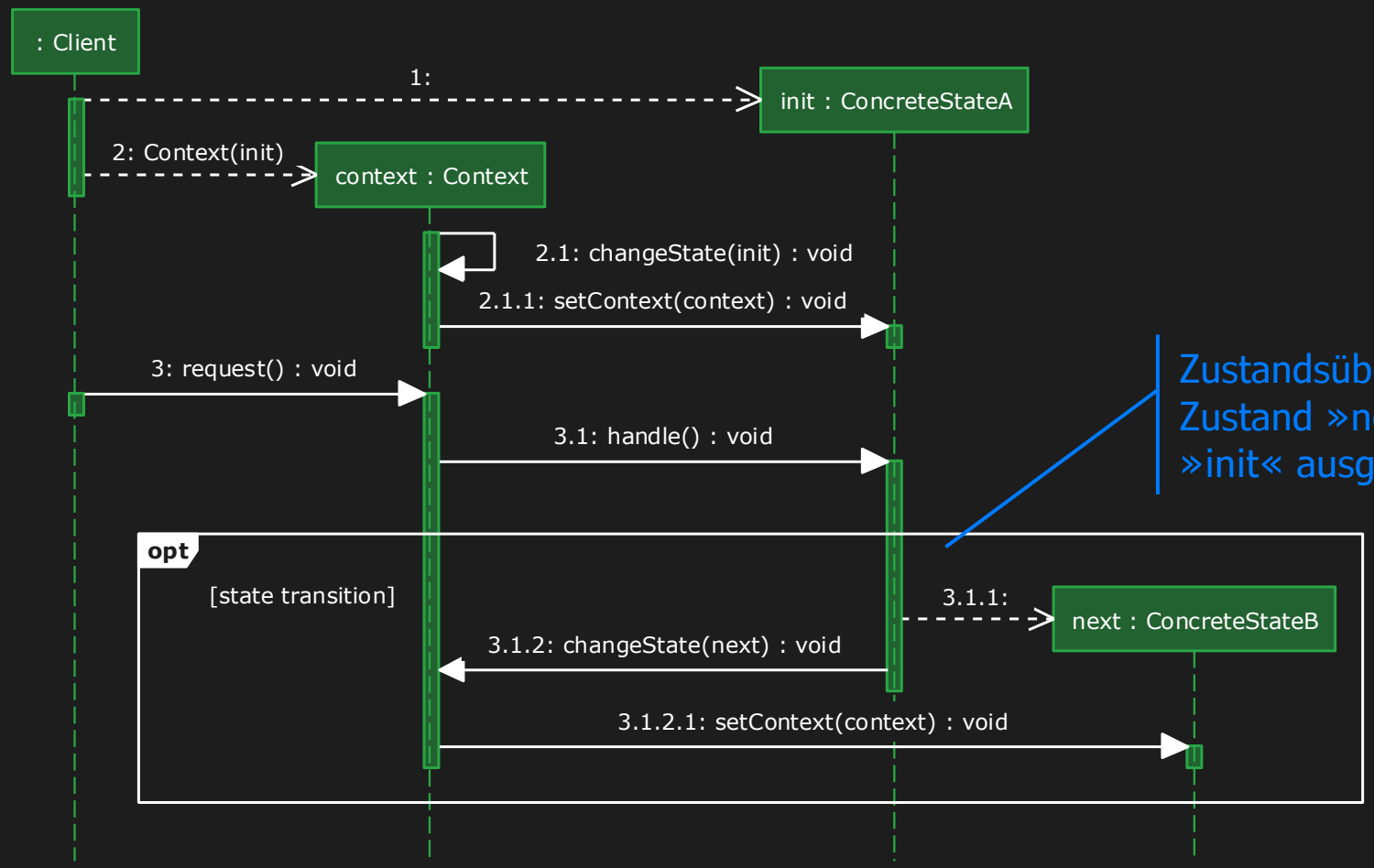
Schnittstelle zur Kapselung von Verhaltensweisen der Zustände



Implementiert die konkrete Verhaltensweise eines Zustands; ist verantwortlich für die eigenen Zustandsübergänge (Transitionen)

- Beachte: durch das Interface wird eine zyklische Abhängigkeit vermieden [→ DIP]
- Attribut `state` von `Context` sollte keinen öffentlichen Getter/Setter haben

State: Sequenz



Zustandsübergang (Transition) zu Zustand »next« wird von Zustand »init« ausgelöst

State: Vor- und Nachteile

Vorteile

- SRP: Verhalten & Code wird durch Zustände in separate Klassen aufgeteilt
- OCP: Neue Zustände können hinzugefügt werden, ohne bestehenden Code zu ändern
- Unerlaubte Zustandsübergänge können besser verhindert werden
- Code wird einfacher und übersichtlicher gegenüber Konstanten, Enums & Switch-Statements

Nachteile

- Jeder Zustand muss sein Verhalten implementieren
- Gemeinsames Verhalten muss für jeden Zustand separat implementiert werden
- Zu viel Overhead, wenn nur wenige Zustände bzw. nur selten Zustands-änderungen

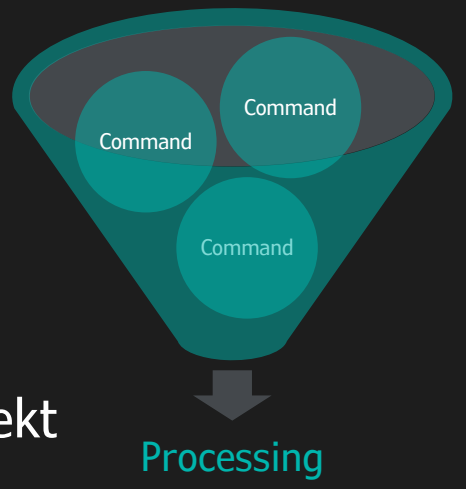
Abschnitt E.4

Command (Befehl)

Verhaltensmuster: Command

Command – Kapselung eines Requests als Objekt, um so die Parametrisierung von Clients mit verschiedenen Requests, Warteschlangen- oder Logging-Operationen sowie das Rückgängigmachen von Operationen zu ermöglichen.

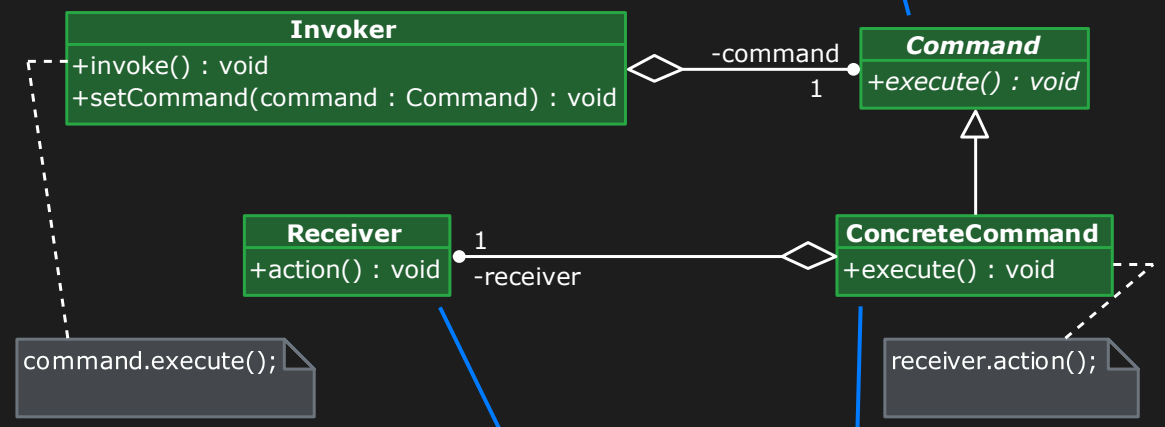
- Anwendungsfälle
  - GUI: gleiche Befehle werden an verschiedenen Orten angeboten (Menü, Kontextmenü, Hotkey)
  - Realisierung einer Rückgängig-Funktion (Undo/Redo)
  - Protokollierung von Befehlen
  - Transaktionssysteme: mehrere Befehle sollen als Einheit vollständig ausgeführt werden, sonst werden sie rückgängig gemacht (»Rollback«)
  - Fortschrittsbalken, Makros, Makro-Recording & Wizards
  - Befehlswarteschlangen & Threadpools
- Befehle
  - Niedrige Intelligenz: keine eigene Logik, nur Delegation an Empfängerobjekt
  - Hohe Intelligenz: umfangreiche Logik

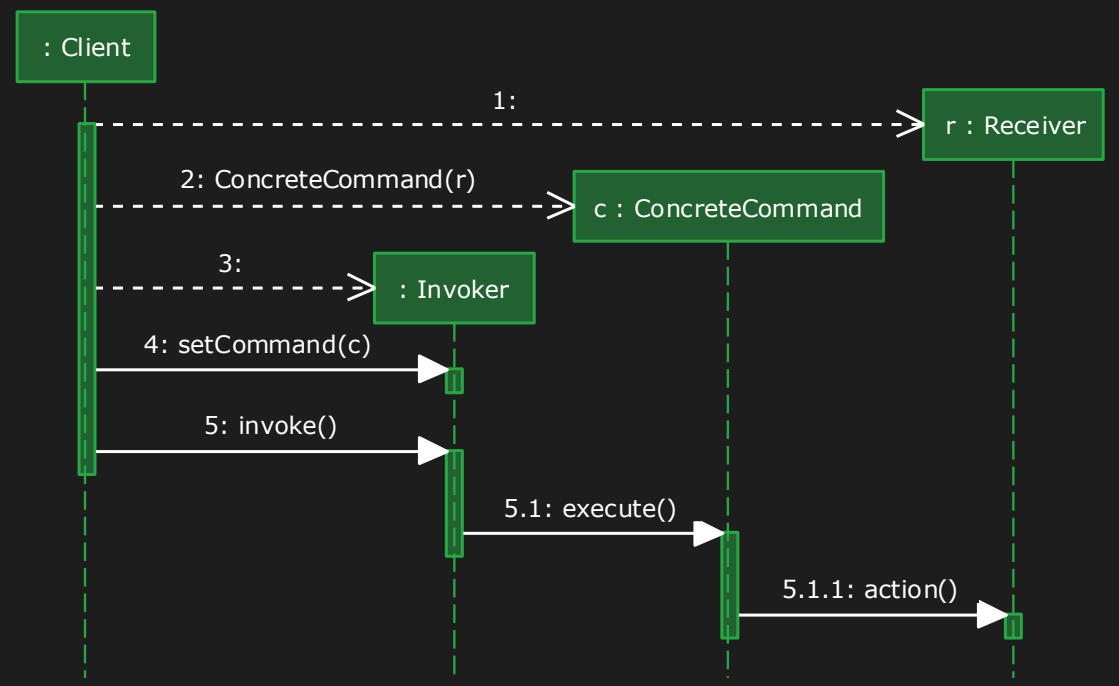


Command: Schema & Sequenz

Weist das Command-Objekt an, den Request auszuführen

Deklariert eine Schnittstelle zur Ausführung einer Operation





Kann die mit der Bearbeitung eines Requests verknüpften Operationen ausführen; kann beliebige Klasse sein

Definiert Bindung zwischen Receiver und Operation; Ausführung durch Aufruf der Operation beim Receiver; kann Parameter der Operation als Attribute aufnehmen & weitergeben

Command: Vor- und Nachteile

Vorteile

- Entkopplung der Objekte, die eine Operation auslösen und diese ausführen [→ SRP]
- Modularität & Wiederverwendbarkeit von Befehlsobjekten
- Realisierung neuer Befehle zur Laufzeit ohne Anpassung von bestehendem Code [→ OCP]
- Kombination von Befehlen (Makros)
- Undo/Redo, Logging, Warteschlangen, Scheduling

Nachteile

- Erhöhte Anzahl von Klassen: jeder Befehl bildet eigene Klasse
- Neue Schicht zwischen Sender & Empfänger (Auslöser & Ausführer von Operationen)

Abschnitt E.5

Memento

Verhaltensmuster: Memento

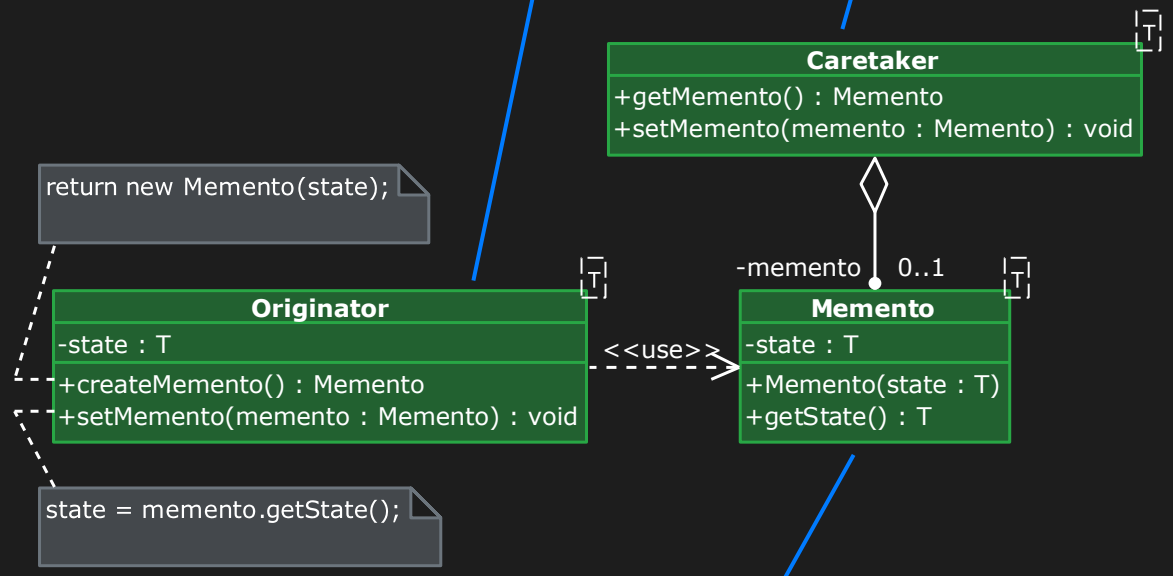
Memento – Erfassung und Externalisierung des internen Zustands eines Objekts, ohne dessen Kapselung zu beeinträchtigen, so dass es später wieder in diesen Zustand zurückversetzt werden kann.

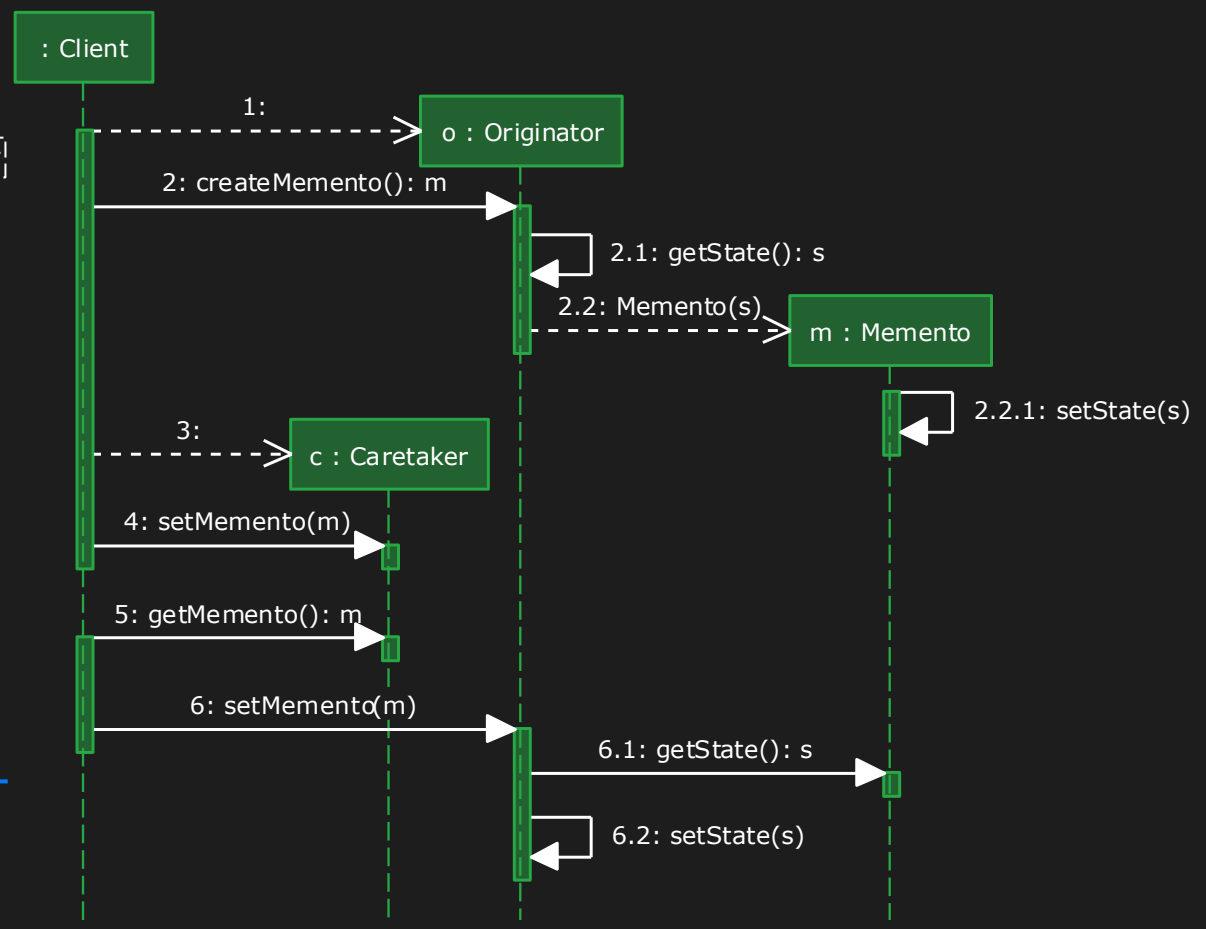
- Anwendungsfälle
  - Speichern des Zustands zur Realisierung einer Wiederherstellen-Funktion (Save / Restore), z.B. auch für die Funktionen Rückgängig und Wiederholen (Undo / Redo) in Kombination mit dem Command-Pattern
  - Wenn eine direkte Schnittstelle zum Abrufen des internen Zustands dessen Kapselung negativ beeinträchtigen würde (z.B. weil Attribute unnötigerweise öffentlich gemacht werden müssten)
- Varianten der Realisierung
  - Memento als eigenständige Klasse
  - Memento als geschachtelte Klasse (Nested Class / Inner Class)

Memento: Schema & Sequenz

Erzeugt Momentaufnahme seines internen Zustands als Memento; stellt bei Bedarf internen Zustand mithilfe eines Mementos wieder her

Verantwortlich für sichere Verwahrung eines Mementos; kann bei Bedarf auch Verlauf als Stapel speichern





Speichert internen Zustand des Originator als Momentaufnahme; schützt Daten vor Änderung / unerlaubtem Zugriff

Memento: Vor- und Nachteile

Vorteile

- Schnappschüsse eines Objekts, ohne die Kapselung zu verletzen
- Vereinfachung des Originator: Caretaker kann z.B. einen Verlauf der Zustände speichern (und nicht Originator)

Nachteile

- Verwendung kann aufwändig und speicherintensiv sein; insbesondere, wenn Clients oft Mementos erzeugen
- Bei dynamischen Sprachen kann nicht ausgeschlossen werden, dass sich der Zustand des Mementos ändert
- Sicherheitskritisch, wenn man nicht ausschließen kann, dass nur Originator auf Memento zugreifen darf

Abschnitt E.6

Observer (Beobachter)

Verhaltensmuster: Observer

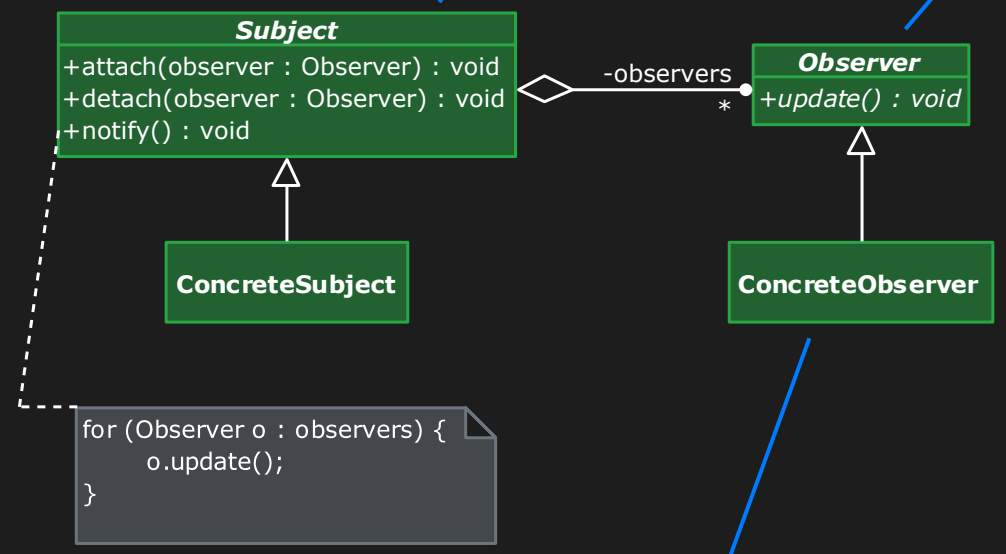
Observer – Definition einer 1-zu-n-Abhängigkeit zwischen Objekten, damit im Fall einer Zustandsänderung eines Objekts alle davon abhängigen Objekte entsprechend benachrichtigt und automatisch aktualisiert werden.

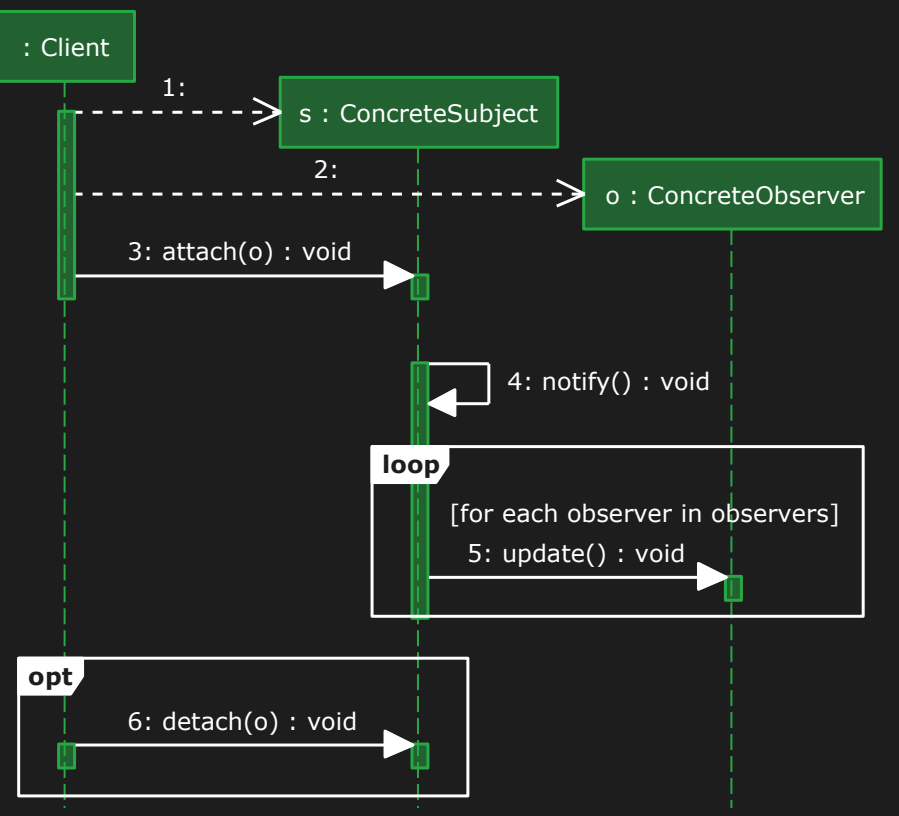
- Anwendungsfälle
  - Die Veränderung eines Objekts macht die Modifikation anderer Objekte nötig, wobei die zu ändernden Objekte zunächst unbekannt sind
  - Objekt sollen andere Objekte benachrichtigen, ohne Details über diese Objekte zu wissen und mit loser Kopplung der Objekte
  - Typisch in GUI-Frameworks, Standard-Implementierung in Java, Event Handling in C# (Schlüsselwort event)
  - Zentrales Entwurfsmuster einer [→ Event-Driven Architecture (EDA)]
- Beispiele
  - Daten in allen GUI-Komponenten aktualisieren, siehe z.B. MVC-Pattern
  - Datensatz mit mehreren Visualisierungen (Multicasts, z.B. Master & Detail Views)

Observer: Schema & Sequenz

Kennt seine Observer; kann von beliebig vielen Observern beobachtet werden; stellt Schnittstelle zum Anbinden und Lösen für Oberver bereit

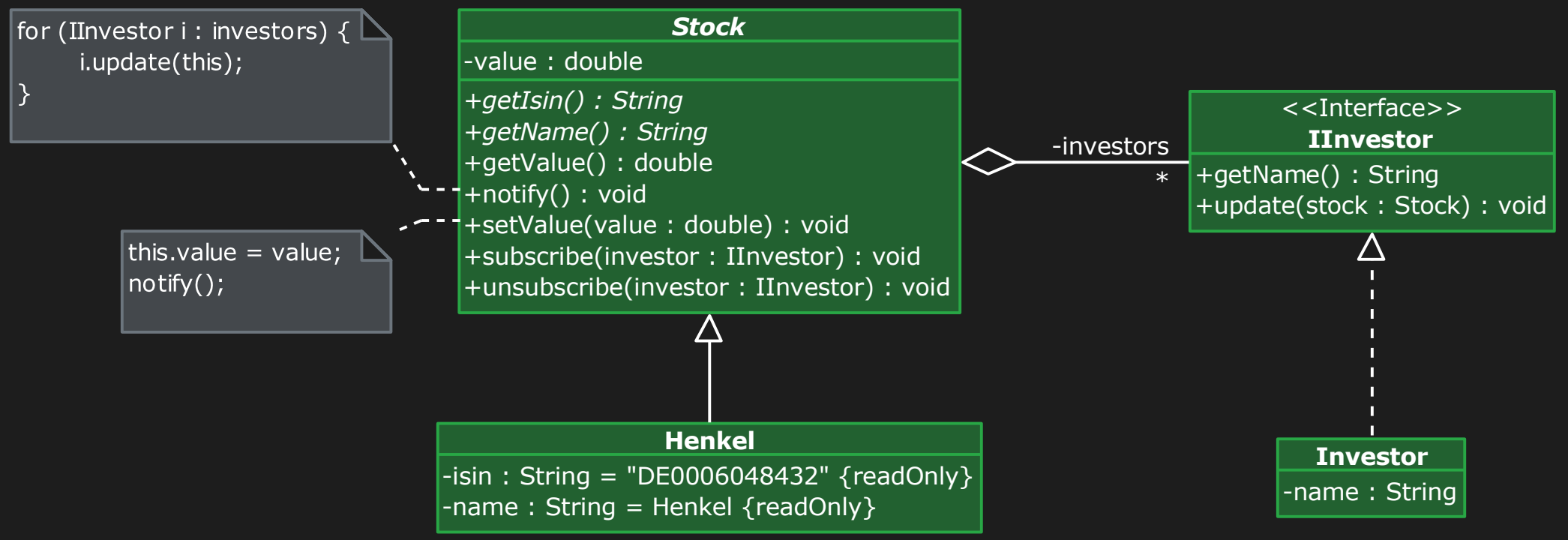
Stellt Schnittstelle für Objekte bereit, die bei Änderungen von Subject benachrichtigt werden sollen





Implementiert die Schnittstelle für die Benachrichtigung über Änderungen von Subject

Observer: Beispiel



Observer: Vor- und Nachteile

Vorteile

- Zustandskonsistenz
- Kopplung nur während des »Abonnement«
- Flexibilität & Modularität
  - Ein Subject, viele Observer
  - Ein Observer, viele Subjects
- Wiederverwendbarkeit
- Kompatibilität zum Schichtenmodell
  - Subject & Observer können in verschiedenen Schichten liegen

Nachteile

- Aktualisierungskaskaden bei vielen Subjects & Observer
- Zyklische Benachrichtigungen sind möglich
- Fehlersuche umständlich
- Mehrfachregistrierung möglich, eventuell Nebeneffekte
- Abmeldung von Observer wird oft vergessen (dadurch keine Garbage Collection)

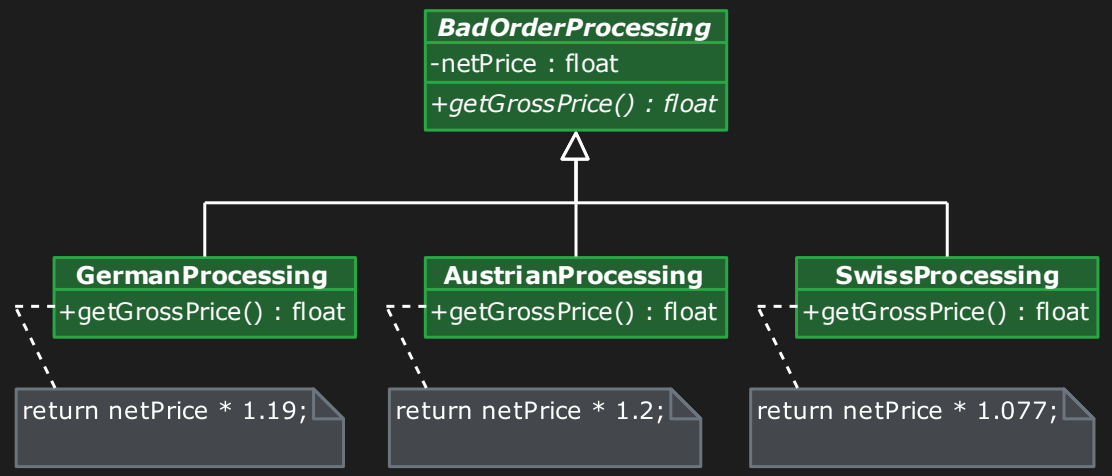
Abschnitt E.7

Strategy (Strategie)

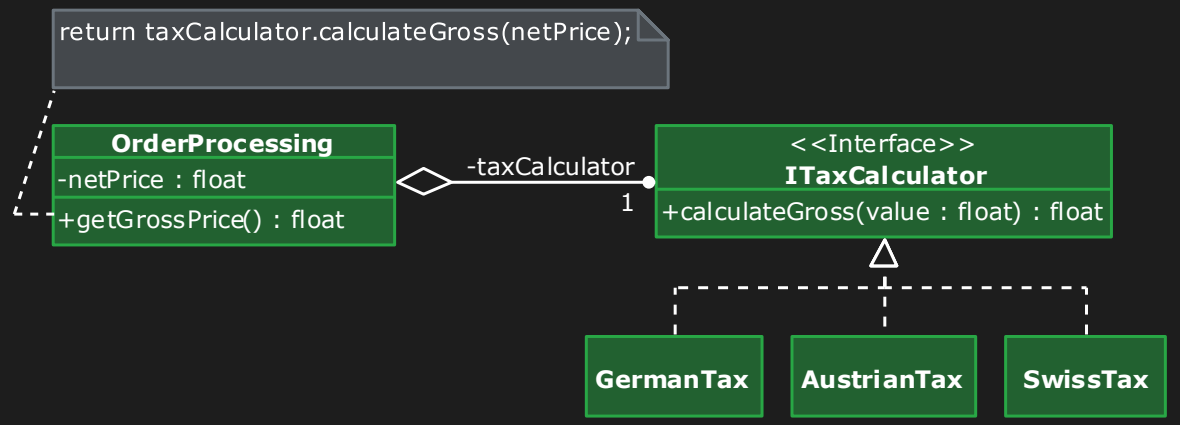
FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Beispiel: Auftragsabwicklung

- Szenario: Anwendung zur elektronischen Auftragsabwicklung
  - Klasse OrderProcessing mit Nettopreis (NetPrice) des Auftrags als Attribut; Bruttopreis (GrossPrice) wird daraus berechnet
  - Anwendung soll für Aufträge aus Österreich & Schweiz erweitert werden
- Steuerberechnung als »Strategie«
  - Steuerberechnungsvorschriften durch Klassen, die ein Interface realisieren
  - Klassen sind nur für diese eine Aufgabe zuständig [→ SRP]
  - Konkrete Vorschrift wird in Klasse OrderProcessing »eingesteckt«



Schlechter Entwurf



Guter Entwurf

Verhaltensmuster: Strategy

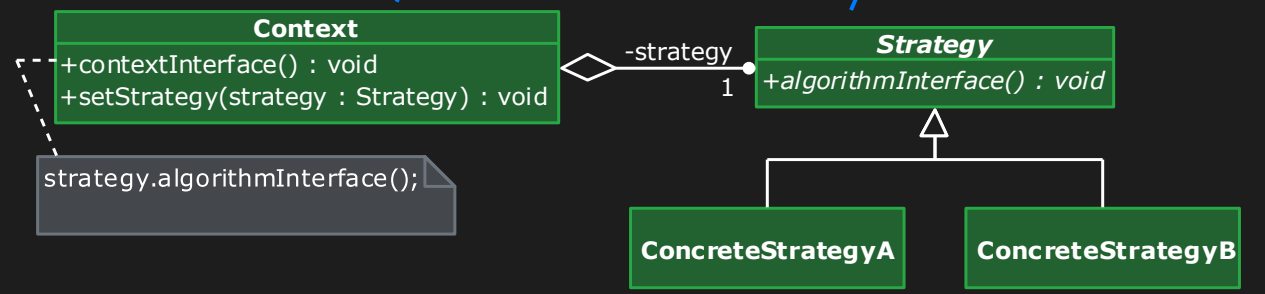
Strategy – Definition einer Familie von einzeln gekapselten, austauschbaren Algorithmen. Es ermöglicht eine variable und von den Clients unabhängige Nutzung des Algorithmus.

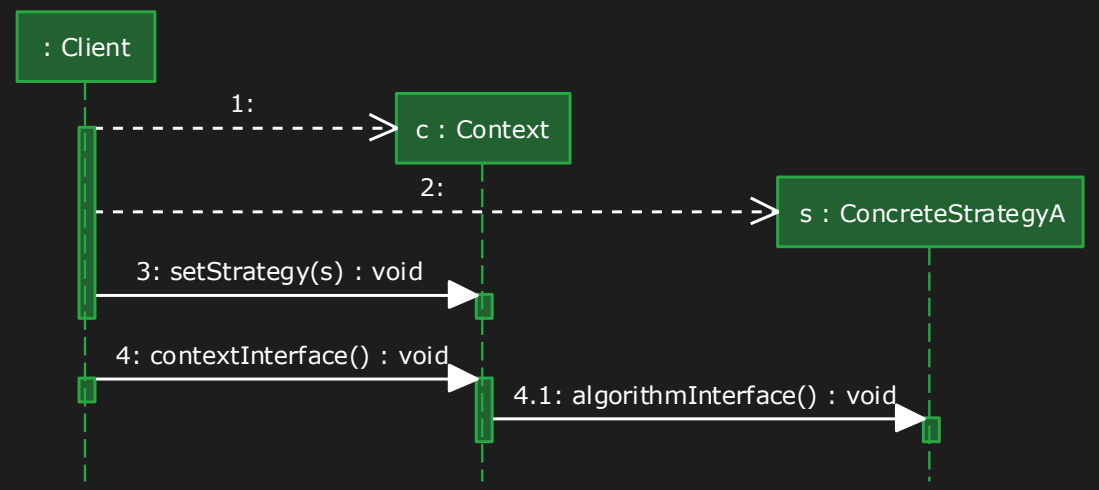
- Problem
  - Verwandte Klassen unterscheiden sich lediglich dadurch, dass sie gleiche Aufgaben teilweise durch verschiedene Algorithmen lösen
- Lösung
  - Die Klassen werden nicht in einer Vererbungshierarchie angeordnet; stattdessen wird eine Klasse erstellt, die alle gemeinsamen Operationen definiert (Context)
  - Die Signaturen der Operationen, die unterschiedlich zu implementieren sind, werden in einem Interface bzw. einer abstrakter Klasse zusammengefasst (Strategy)
  - Jede Implementierung ConcreteStrategy realisiert Strategy
  - Context benutzt die ConcreteStrategy-Objekte, um die unterschiedlich implementierten Operationen durch Delegation auszuführen

Strategy: Schema & Sequenz

Besitzt die Referenz auf genau ein Strategy-Objekt

Definiert gemeinsame Schnittstelle für alle Algorithmen





Implementiert Variationen des Algorithmus mittels der Strategy-Schnittstelle

Strategy: Vor- und Nachteile

Vorteile

- Alternative zur Unterklassenbildung, erfüllt Favour Composition over Inheritance (FCoI)
- Wiederverwendbarkeit durch Entkopplung von Kontext & Verhalten
- Dynamisches Verhalten, Auswahl von Algorithmen zur Laufzeit
- Vermeidung von Fallunterscheidungen / Bedingungen

Nachteile

- Enge Kopplung zwischen Client & Strategieimplementierung: Client muss ConcreteStrategy kennen und in Context setzen (Abhilfe: Factory Method)
- Unnötiger Datentransfer zwischen Context und Strategy, wenn ConcreteStrategy nicht alle Parameter von Strategy benötigt

Abschnitt E.8

Decorator (Dekorierer)

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Strukturmuster: Decorator

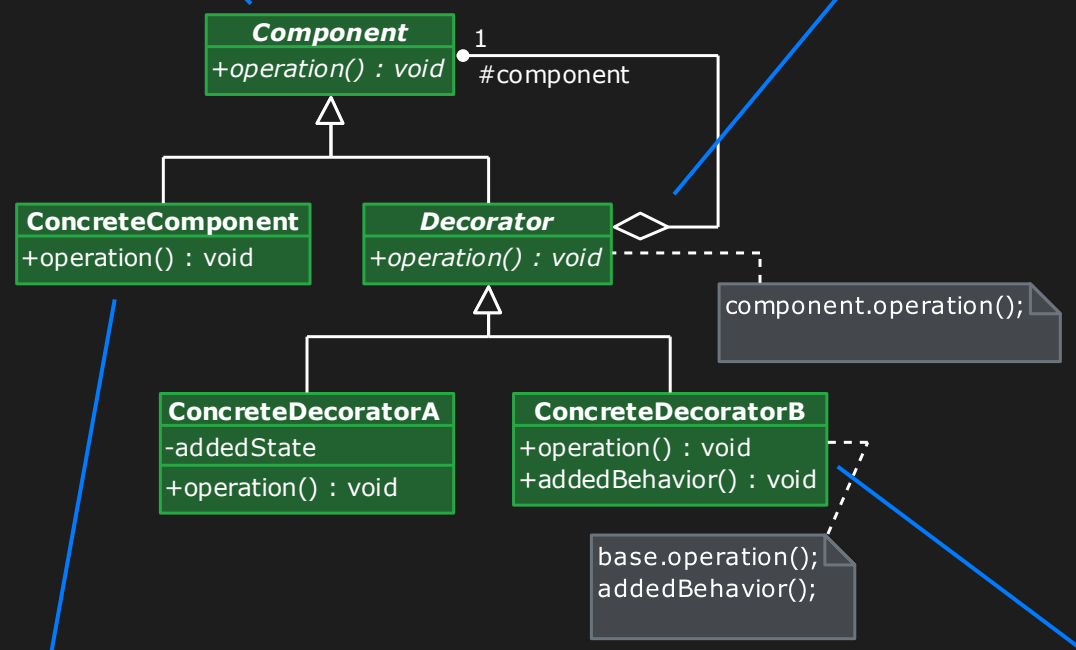
Decorator – Dynamische Erweiterung der Funktionalität eines Objekts. Decorator-Objekte stellen hinsichtlich der Ergänzung einer Klasse um weitere Zuständigkeiten eine flexible Alternative zur Unterklassenbildung dar.

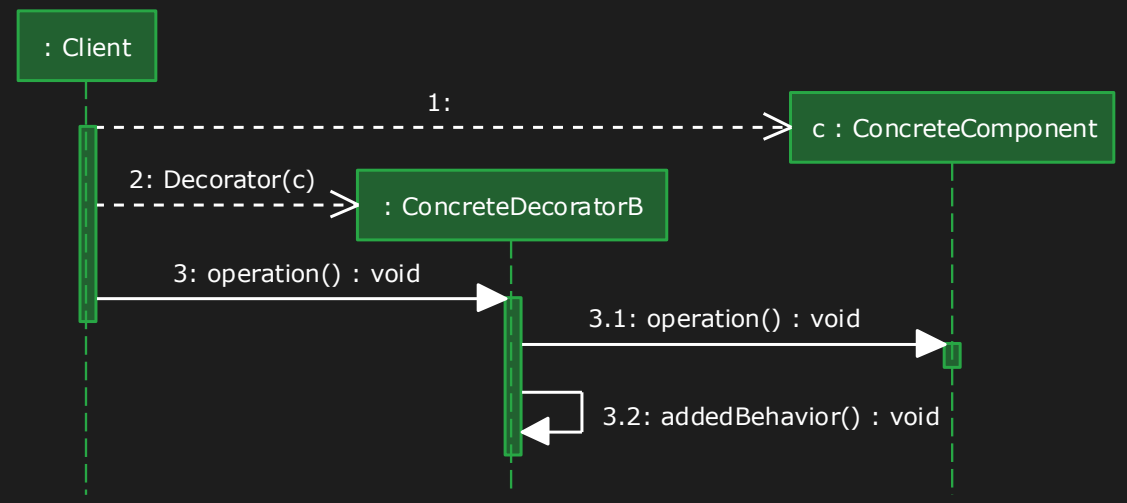
- Anwendungsfälle
  - Wenn einzelne Objekte dynamisch und transparent mit weiterer (weitgehend unabhängiger) Funktionalität ausgestattet werden sollen, ohne dass andere Objekte davon betroffen sind
  - Wenn ergänzte Funktionalität auch wieder entfernt werden kann
  - Wenn die Erweiterung mittels Vererbung nicht praktikabel ist, weil zu viele Kombinationen entstehen oder weil eine Klasse keine Vererbung ermöglicht (final bzw. sealed)
- Beispiel
  - Essensauswahl mit drei Hauptgerichten & vier Beilagen: eine einfache Vererbung würde 12 Unterklassen benötigen und erlaubt keine Kombination der Beilagen

Decorator: Schema & Sequenz

Definiert Schnittstelle für Objekte, die dynamisch um zusätzliche Funktionalität erweitert werden können

Enthält Referenz auf ein Component-Objekt & definiert Schnittstelle, die der Component-Schnittstelle entspricht



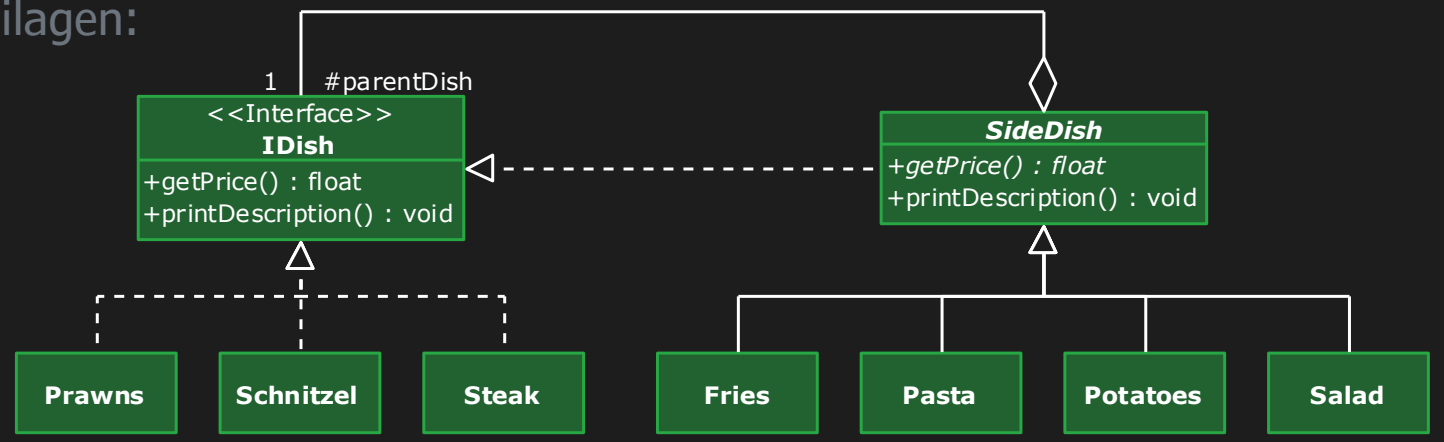


Definiert Objekt, das um zusätzliche Funktionalität erweitert werden kann

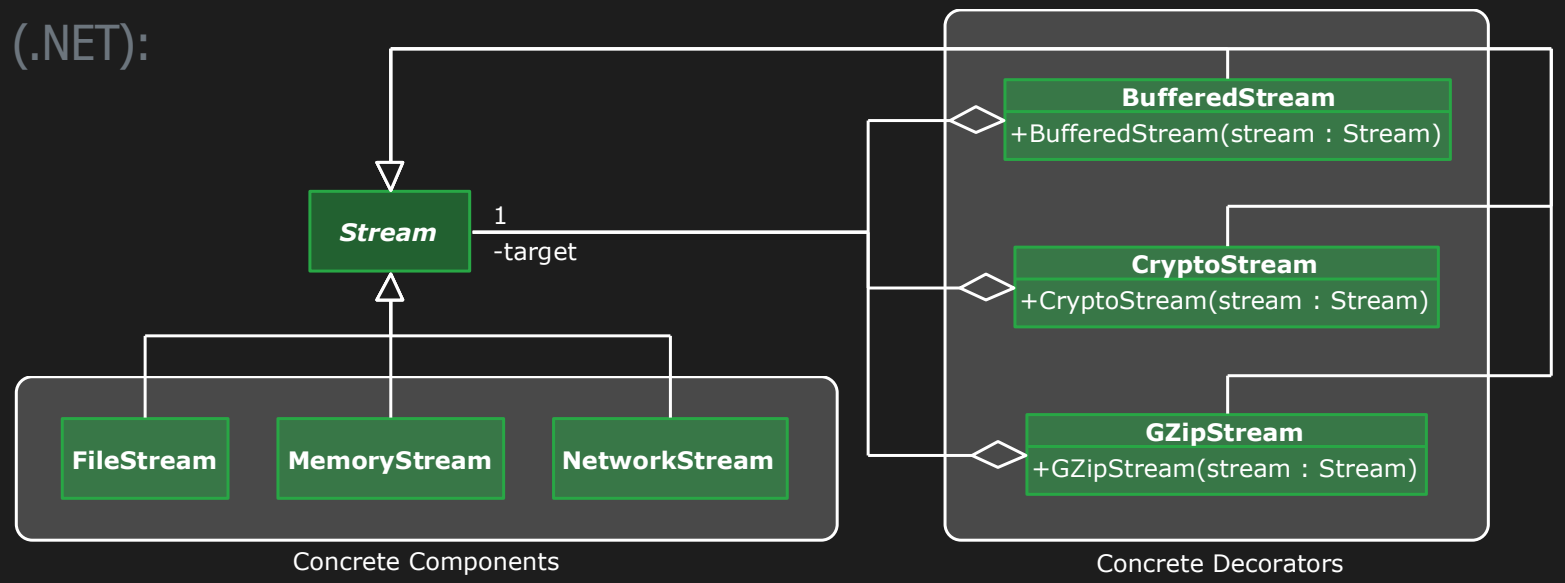
Stattet die Komponente mit zusätzlicher Funktionalität aus

Decorator: Beispiele

Hauptgerichte mit Beilagen:



System.IO.Stream (.NET):



Decorator: Vor- und Nachteile

Vorteile

- Flexible und transparente Funktionserweiterungen
- Hinzufügen oder Entfernen von Funktionalität / Verantwortlichkeit zur Laufzeit
- Keine Überfrachtung von Klassen, nur notwendige Erweiterungen [→ SRP]
- Kombinierbarkeit von Erweiterungen, auch mehrfach
- Wartbarkeit durch schlanke, kohäsive Klassen

Nachteile

- Hohe Objektanzahl: für jede Erweiterung ein neues Objekt
- Unkomfortable, wortreiche API: die Objektkonstruktion ist nur schwer nachvollziehbar
- Keine Objektidentität zwischen ursprünglicher Komponente & dekorierter Komponente
- Erschwerte Fehlerfindung